

MACRO PROPERTY SERVICES

Architecture Document

Employee App — React Native Edition (Proposed)

Proposed change

React Native (Expo)

Same Supabase + Firebase backend

Admin Dashboard unchanged

01 What's Changing

This replaces the Employee App's implementation only — from a Next.js mobile-first PWA to a real React Native app (installed from the App Store / Play Store, or side-loaded during development). Nothing else moves: the Admin Dashboard stays a Next.js web app, and the entire backend — Supabase Postgres, Row-Level Security, Firebase Firestore chat, image hosting — is reused exactly as it is today. This is a client rewrite, not a backend migration.

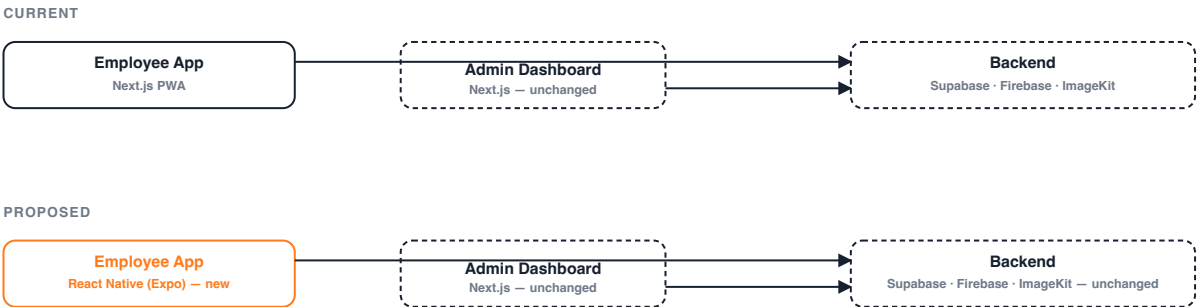


Fig. 1 — Only the Employee App box changes. It talks to the same backend the same way in spirit — direct calls to Supabase and Firebase, no new custom API server introduced.

Why move off the PWA

Real access to native device APIs (background location, native camera, reliable push notifications even when the app is fully closed) and a proper App Store / Play Store presence — none of which a browser-based PWA can fully guarantee across devices.

What stays identical

Every table, RLS policy, Edge Function, and Firestore rule described in the main Architecture Document is reused unchanged. The Admin Dashboard needs no changes at all.

02 Tech Stack — Employee App

Recommending Expo's managed workflow over bare React Native — this team has no existing native (Xcode/Android Studio) tooling investment, and Expo removes that entirely for a CRUD-plus-camera-plus-location app like this one.

LAYER	CHOICE	WHY
Framework	React Native + Expo (managed)	No local Xcode/Android Studio build setup required for day-to-day development; EAS Build handles native compilation in the cloud.
Navigation	Expo Router	File-based routing, closest mental model to the existing Next.js App Router route structure — the current sitemap maps over almost directly.
Auth session storage	Supabase JS SDK + AsyncStorage	Replaces the cookie-based session (<code>@supabase/ssr</code>) the web app uses — there are no cookies on a native client, so the session token is persisted to on-device storage instead.
Location	expo-location	Replaces the browser Geolocation API; same 20m-geofence Haversine check, re-validated server-side exactly as today.
Camera / photo picker	expo-image-picker / expo-camera	Replaces the web <code><input type="file"></code> flow for checklist/audit/communication photos.
Push notifications	expo-notifications (FCM under the hood)	Native push — more reliable than the current web-push/VAPID approach, and works even when the app is fully closed.
Live chat	Firebase JS SDK (unchanged)	Same Firestore project, same <code>mint-firebase-token</code> custom-token bridge — works from React Native as-is.
Styling	React Native StyleSheet + shared design tokens	Tailwind's utility classes don't apply to native views — colors/spacing/type scale from <code>packages/shared/src/tokens</code> are still reused as raw values.
Build & release	EAS Build / Submit / Update	Cloud builds for both platforms, store submission, and over-the-air JS updates without a full store review cycle for most changes.

03 Shared Code Strategy

`packages/shared` currently assumes a Next.js/browser environment in a few specific places. Splitting it by platform keeps the genuinely cross-platform logic — the parts most worth not duplicating — shared, while giving each runtime its own client setup.

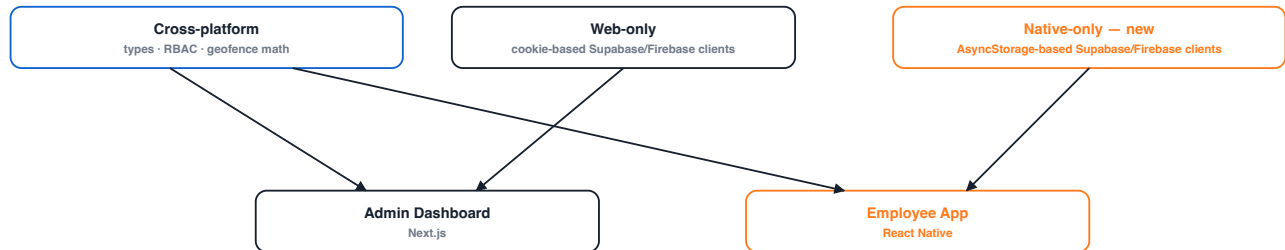


Fig. 2 — Types, the RBAC permission schema/helpers, and the geofence Haversine formula move to both apps unchanged.

Each app gets its own Supabase/Firebase client setup, since session storage and network primitives differ between a browser and a native runtime.

What ports over unchanged

- All TypeScript row types (`Company` , `Employee` , `Checklist` , etc.)
- `RolePermissions` shape and every RBAC helper (`canViewAppArea` , `firstAvailableAppRoute` , ...)
- Geofence distance math (`haversineDistanceMeters` , `withinGeofence`)
- Design tokens as raw values (colors, spacing, radii) — just consumed differently (StyleSheet vs Tailwind)

What needs a native-specific version

- Supabase client — `createClient()` with an AsyncStorage adapter instead of `@supabase/ssr` cookies
- Firebase client init and `ensureFirebaseSession()` — same custom-token flow, native SDK
- Every UI component — React Native has no DOM, so Tailwind classes and web components (`<div>` , `<input>`) don't carry over; screens are rebuilt with native primitives

04 Backend — Unchanged

The React Native app talks to the exact same backend surface as the current PWA, using the same Row-Level Security policies, the same Edge Functions, and the same Firestore rules. Nothing here needs to be redeployed or modified for the client rewrite.

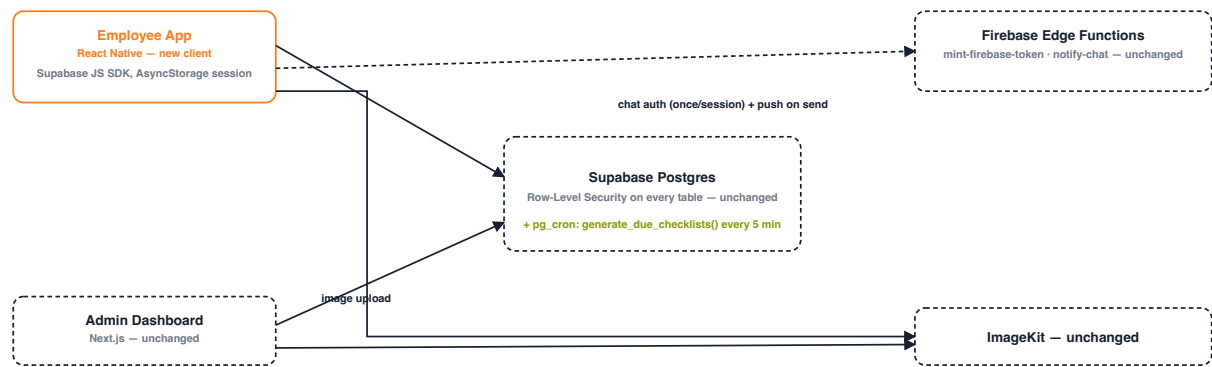


Fig. 3 — Same diagram as the main Architecture Document's backend flow, with the Employee App's implementation swapped out. Every box on the right is reused exactly as deployed today.

05 Auth, Geofencing & Push — What Actually Changes Under the Hood

Authentication & session persistence

Login is still Supabase Auth email/password — same credentials, same `roles` / `employees` lookup to decide app access. The difference is *where the session lives*: the web app stores it in an httpOnly cookie via `@supabase/ssr`; the native app has no cookies, so the Supabase client is configured with an `AsyncStorage`-backed storage adapter instead, and every request carries the token explicitly rather than relying on the browser to attach a cookie automatically.

Geofencing

`expo-location` replaces the browser's Geolocation API for getting a GPS fix at Clock In/Out. The distance math itself — Haversine, 20m default radius — is untouched and reused directly from `packages/shared/src/geo`. Native location access is generally more reliable than a mobile browser's, particularly for background/foreground accuracy prompts.

Push notifications

This is a genuine improvement, not just a port. The current web-push setup depends on a VAPID key and the browser tab/PWA staying registered with the OS's push service, which is inconsistent across devices when the PWA isn't actively running. Native FCM via `expo-notifications` is the standard, reliable channel on both iOS and Android — the existing `notify-chat` Edge Function and `device_tokens` table need no changes; only the token registered per device changes shape slightly (an Expo/FCM push token instead of a Web Push subscription).

Net effect: geofencing and push notifications — the two features most limited by running inside a mobile browser — both get more reliable, not just equivalent, on React Native.

06 Build & Distribution

Replaces "push to Vercel" for this one app with a store-based release process — a real shift in how updates reach employees.

TODAY (PWA)	PROPOSED (REACT NATIVE)
Deploy to Vercel; employees get the update on next page load.	<code>eas update</code> pushes most JS/UI changes over-the-air, often within minutes — no store review for these.
No app store presence; installed via "Add to Home Screen."	<code>eas submit</code> to the Apple App Store and Google Play — required once for any native-code change (new native module, permission, icon).
Single build target (the web).	<code>eas build</code> produces separate iOS and Android binaries from one codebase.

Trade-off to plan for: native-code changes (adding a permission, a new native module, an icon/splash update) still require App Store / Play Store review — typically hours to a couple of days on Apple's side. Day-to-day feature/bugfix work should stay JS-only where possible to keep using instant OTA updates.

07 Migration Notes & Open Decisions

Suggested approach

- New app lives at `apps/employee-app-native` alongside the existing `apps/employee-app` — build it out screen-by-screen against the live backend before cutting employees over, rather than a big-bang rewrite.
- Port screens in the same order as the current sitemap: Login → Home → Attendance → Checklists → Audits → Communication → Profile.
- Once feature-complete and tested on both platforms, retire `apps/employee-app` (the PWA) and its Vercel deployment.

Decisions still open

QUESTION	WHY IT MATTERS
Expo managed workflow vs. bare React Native?	This document assumes managed Expo (recommended, no native tooling needed). Bare RN would be justified only by a specific native module Expo can't support.
Any offline support needed in the field?	Not present in the current PWA either, but native apps make on-device caching/offline-queue far more practical if sites often have poor signal.
Minimum supported OS versions?	Affects which Expo SDK features (e.g. certain camera/location APIs) are safe to rely on.
Internal distribution during development?	EAS supports internal test builds (ad-hoc/TestFlight-style) before any public store submission — worth setting up early for employee testing.

Macro Property Services — Architecture Document (React Native Employee App, proposed). Pairs with the main Architecture Document, which this doesn't replace — the backend sections there still apply in full.